Group Details

Course: AIMLZG 549 - API-Driven Cloud-Native Solutions
Assignment: Assignment II **Institution:** BITS Pilani, Work Integrated Learning Programmes
Group No.: 49 **Source repository:** GitHub
Demonstration video: Watch the project demonstration

Group Member Names with Contribution

We shared the work across the project and report an equal contribution of 25% for each of the four members. The descriptions below identify each member's main tasks; they are not an assignment of one whole assessment part to one person.

Sl. No.	BITS ID	Name	Contribution of team member (Qualitative)	Percentage Contribution (Out of 100) Quantitative
1	2025 ae05 391	Bhuvnesh Mishra	Developed the React interface and FastAPI routes; integrated classification, handoff summaries, and RAG question answering in one workflow.	25%
2	2025 ae05 828	Chezrla Raga Suma	Prepared and reviewed classification messages and labels; fine-tuned the FLAN-T5 classifier and evaluated it against the base model.	25%
3	2025 ae05 425	Sathish Krishnan V	Selected and checked the WHO source, prepared the local knowledge passages, and implemented the embedding retrieval used for question answering.	25%
4	2025 ae05 898	Pushadapu Sanjay Kumar	Defined the clinic-support workflow, prepared representative synthetic cases, and analysed the results against workplace needs.	25%
Total				**100%**

Diabetes Care Support Assistant

Our team built an interactive application for a diabetes-clinic support desk. This report follows the four parts of Assignment II: the API-based application, fine-tuning, retrieval-augmented generation, and workplace application.

Project and Platform Details

Selected application category: Natural Language Processing (NLP). We chose healthcare support for a diabetes-clinic desk. Staff can enter a message or ask a question; the app returns a category, a handoff summary, or an answer with its source passages. We connected three NLP tasks in one React and FastAPI workflow: text classification, text summarisation, and question answering using retrieval-augmented generation (RAG).

NLP task	Natural-language input and output	Method/model
Text classification	Support message → one of five clinic-support labels	Supervised fine-tuning of `google/flan-t5-base`
Text summarisation	Support message → staff handoff summary	`google/flan-t5-base` text generation
Question answering using RAG	Staff question → answer grounded in relevant WHO passages, returned with sources	`sentence-transformers/all-MiniLM-L6-v2` retrieval and FLAN-T5 generation

We pinned the FLAN-T5 base model to revision `7bcac572ce56db69c1ea7c8af255c5d7c9672fc2`. The classifier runs separately; summarisation and answer generation share a FLAN-T5 runtime. Retrieval uses its configured embedding model. Our API exposes `/health`, `/metadata`, `/analyze`, `/retrieve`, and `/chat`.

Architecture Overview

In our app, React sends JSON requests to FastAPI and displays the returned JSON. The model and retrieval components run within the FastAPI application. `/analyze` returns a category and handoff summary, `/retrieve` returns ranked WHO passages, and `/chat` checks for urgent or personal-care questions before it generates a source-grounded answer. The classifier has its own fine-tuned runtime; summarisation and answer generation share the base FLAN-T5 runtime.

```
flowchart LR
    UI[React interface] -->|JSON HTTP request| API
    API -->|JSON HTTP response| UI
    subgraph Backend[FastAPI application process]
        API[Request validation and route handlers]
        API --> A[POST /analyze]
        API --> R[POST /retrieve]
        API --> C[POST /chat]
        API --> S[GET /health and /metadata]
        A --> Classifier[Fine-tuned classifier runtime]
        A --> Base[Shared base FLAN-T5 runtime]
        R --> Retriever[Shared embedding retriever]
        C --> Checks[Routing and answer checks]
        Checks --> Retriever
        Checks --> Base
        WHO[Local WHO passages] --> Retriever
    end
```

The arrows within FastAPI show Python component calls and data dependencies. `/analyze` combines the classifier output and summary; `/retrieve` returns ranked passages; `/chat` returns an answer with passages or a fixed referral response. The following flow expands the chat checks implemented by the knowledge answerer.

```
flowchart LR
    Q[Validated question] --> U{Urgent pattern matched?}
    U -->|Yes| Urgent[Fixed urgent-care response]
    U -->|No| P{Personal or clinic request?}
    P -->|Yes| Referral[Staff referral with no sources]
    P -->|No| R[Retrieve top three WHO passages]
    R --> Score{Passages pass score check?}
    Score -->|No| Referral
    Score -->|Yes| G[Generate using question and retrieved context]
    G --> Valid{Output passes format checks?}
    Valid -->|No| Referral
    Valid -->|Yes| Answer[Answer with retrieved passages]
```

The score check requires at least one passage and a top similarity score at or above the configured threshold. The output check rejects empty text, an ABSTAIN prefix, and citation-only text. These checks handle the listed conditions; they do not establish that every generated statement is supported. Answer support is assessed separately against the retrieved text.

I. API-Based AI Application

Domain and workflow

We modelled a diabetes-clinic support desk. Staff read incoming messages, route requests, prepare handoffs, and consult education material for general questions. In the app, they review the assistant's output before using it.

For example, a message can ask to change an appointment and find an education leaflet. `/analyze` returns a category and handoff summary; we evaluate the summaries in Part II. Staff can also ask a general diabetes question in the knowledge assistant and inspect its answer and source passages.

Application and API

The React interface collects support messages and knowledge questions. FastAPI validates each request and calls the configured models or retriever. `POST /analyze` returns a category and summary, `POST /retrieve` returns ranked passages, and `POST /chat` returns an answer, staff referral, or urgent-care response. `/metadata` shows which components and model or index versions are loaded.

The classifier uses the Hugging Face Transformers sequence-to-sequence interface. The application loads the tokenizer and model from the configured Hub model or local checkpoint. Classification uses a fixed task instruction and returns the generated label only when it matches one of the five supported categories: appointment support, education resource request, portal technical support, records/insurance administration, or clinical staff referral.

API demonstration

The Swagger UI request below demonstrates `POST /analyze` with a synthetic rescheduling request. The React screenshot shows the category and handoff-summary field in the interactive application.

Figure 1. Swagger request and response for `POST /analyze`.

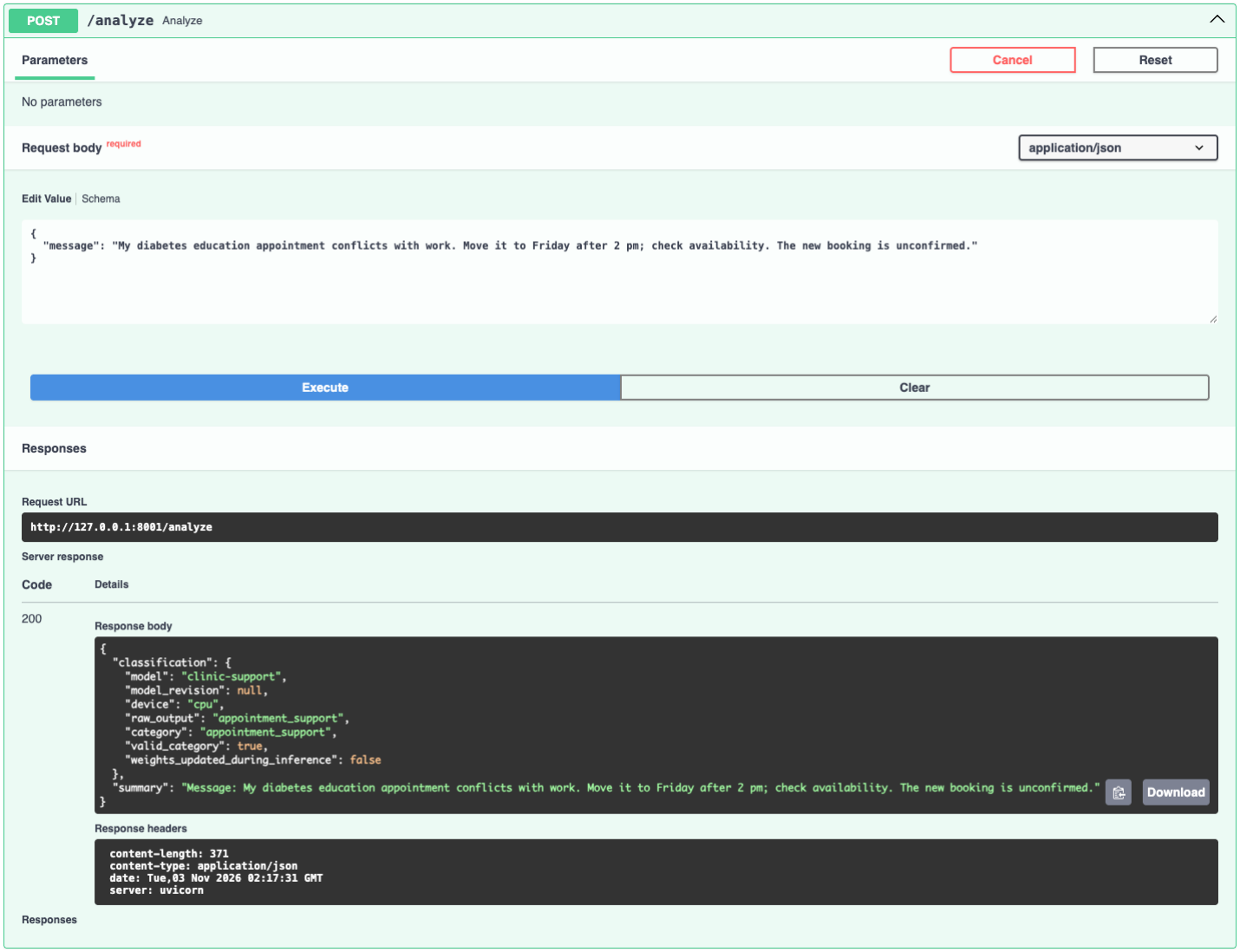


Figure 1: Swagger request and successful response for support message analysis

Figure 2. Live support-message analysis in the interactive application.

Functional validation covered support-message analysis, question answering with displayed sources, and an urgent-care response. The frontend production build succeeded.

II. Fine-Tuning

Dataset and method

The classifier uses five support categories. The dataset contains 280 synthetic training messages, 60 validation messages, and 60 held-out test messages, with 56 training and 12 validation/test examples per category. The messages are AI-drafted

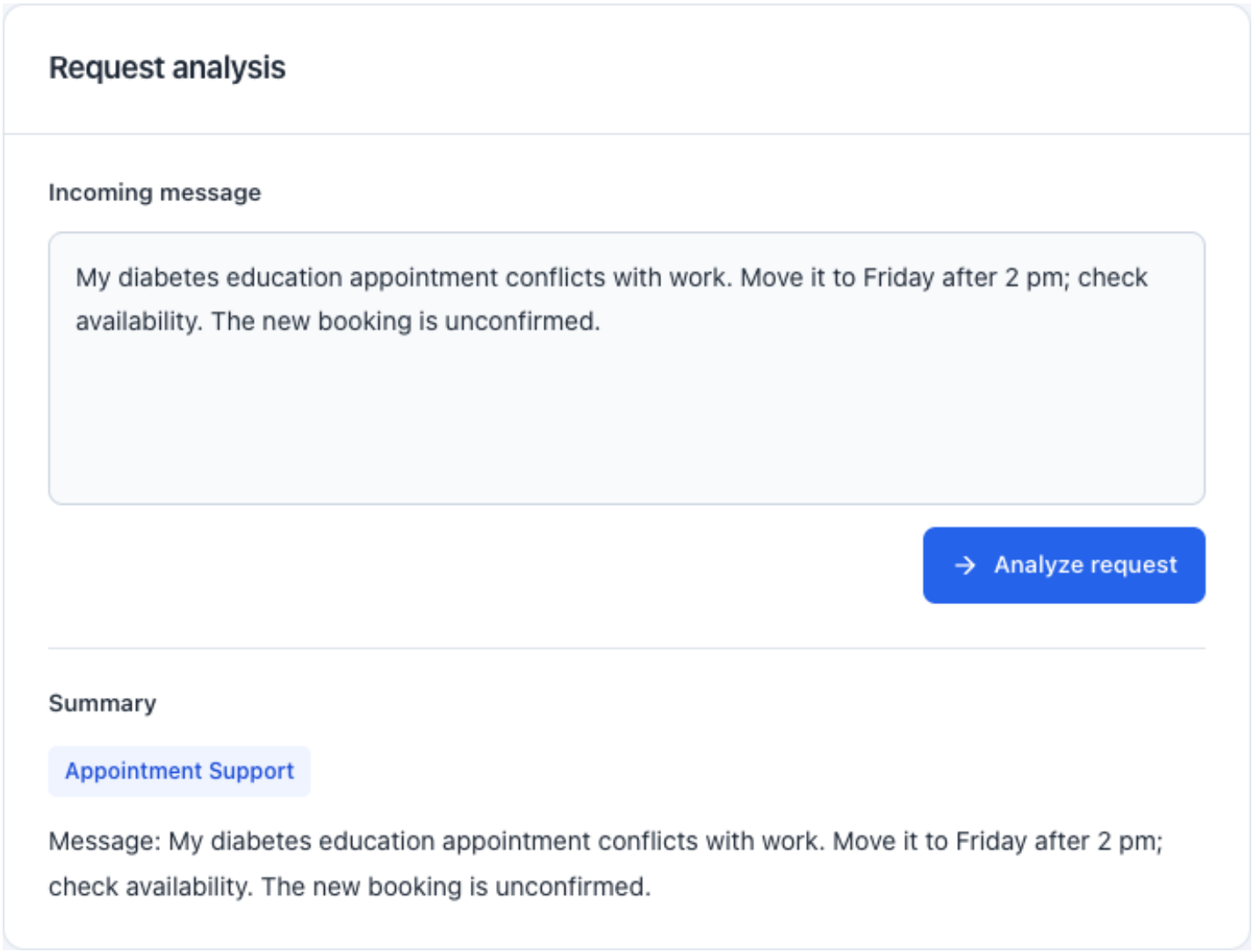


Figure 2: Interactive application showing a support request, its category, and handoff summary

and reviewed for the classification task. Scenario-family identifiers were used when preparing the partitions, and the exact-duplicate review found no repeated messages across the partitions.

We started with FLAN-T5 because it can follow instructions and generate text. The app needs it to return one of five clinic-support labels, including a referral label for personal clinical questions. A general model does not know our label guide, so we fine-tuned it using an instruction and support message as input and the reviewed category as the target. This taught the model to map our example messages to the clinic's labels.

We fully fine-tuned the pinned `google/flan-t5-base` checkpoint. For this run, AdamW updated the model for one epoch with batch size 1, gradient accumulation 8, learning rate `5e-5`, and seed 42; it completed 35 optimizer steps. We saved the selected checkpoint locally and used it in the evaluated app. To reproduce the run, install the inference dependencies and, from `Assignment-2`, run `uv run --extra inference python scripts/train_classifier.py --epochs 1 --output-dir artifacts/models/clinic-support-candidate`. Set `CLASSIFIER_MODEL_NAME=./artifacts/models/clinic-support-candidate` in `.env` and restart the API to load it. The training script uses the tracked training and validation data, not the held-out test set. Evaluate a new checkpoint separately before comparing it with the scores below. The comparison measures our task-specific fine-tuning against the prompted base model.

Validation and held-out results

The fine-tuned checkpoint achieved 98.33% accuracy and 98.33% macro-F1 on the 60-example validation set. The validation set was used during model development, so these figures are validation results. The held-out test comparison below is the reported base-versus-tuned comparison.

The base and fine-tuned models were then evaluated on the same 60-example held-out test set, with the same application prompt and greedy decoding. The test set was not used for training or model selection.

Measure	Base model	Fine-tuned classifier
Accuracy	86.67% (52/60)	95.00% (57/60)
Macro precision	95.00%	95.39%
Macro recall	86.67%	95.00%
Macro-F1	88.74%	94.74%
Invalid category outputs	4/60	0/60

The examples and test set are synthetic; these scores summarise classification on the 60-message set.

Figure 3. Fine-tuning method and held-out classifier results.

Summary quality check

The configured base summariser was evaluated on 20 reviewed synthetic message/summary cases with essential-fact checklists. Mean ROUGE-L F1 was 0.3912. All 20 outputs were supported by their input messages, and 10 of 20 retained every checklist fact.

III. Retrieval-Augmented Generation

Source and retrieval

We prepared the indexed corpus from one public source: WHO's *Diabetes: Questions and Answers* (https://www.who.int/news-room/questions-and-answers/item/diabetes), dated 1 September 2026 in our local source manifest. It contains eight curated passages about diabetes basics, common symptoms, prevention, and general education and self-management support. We kept the title, section, URL, and chunk ID with each passage.

We pinned the retriever, `sentence-transformers/all-MiniLM-L6-v2`, to revision `1110a243fdf4706b3f48f1d95db1a4f5529b4d41`. It embeds the question and WHO passages, ranks them by cosine similarity, and returns the top three. For questions that proceed to retrieval, the `/chat` handler passes those passages with the question to FLAN-T5 and returns the answer with its sources for staff to check. The separate `/retrieve` endpoint returns ranked passages without generating an answer. This lets us update the source material without fine-tuning the answer model. Personal-care and clinic-process questions get a staff referral, and selected urgent phrases get a fixed urgent-care response. Other questions go through the retrieval-score and output-format checks shown in the architecture flow.

PART II · FINE-TUNING

Clinic support classifier

google/flan-t5-base → fine-tuned clinic-support checkpoint

TRAINING METHOD

Full fine-tuning

AdamW · 1 epoch · 35 optimizer steps

DOMAIN DATA

280 messages

5 categories · 60 validation · separate 60-message test set

TRAINING SETTINGS

Batch 1 × 8

Gradient accumulation · learning rate 5e-05

Held-out test results

Measure	Base FLAN-T5	Fine-tuned classifier
Accuracy	86.67%	**95.00%**
Macro-F1	88.74%	**94.74%**
Invalid category outputs	4/60	**0/60**

Test data was held out from training and checkpoint selection. Results are for synthetic clinic-support messages.

Figure 3: Full fine-tuning setup and base versus tuned held-out metrics

API demonstration

Swagger UI confirms that `/retrieve` returns ranked WHO passages and that `/chat` returns a staff referral for an individual medication question.

Figure 4. Swagger response from `POST /retrieve` with ranked WHO passages.

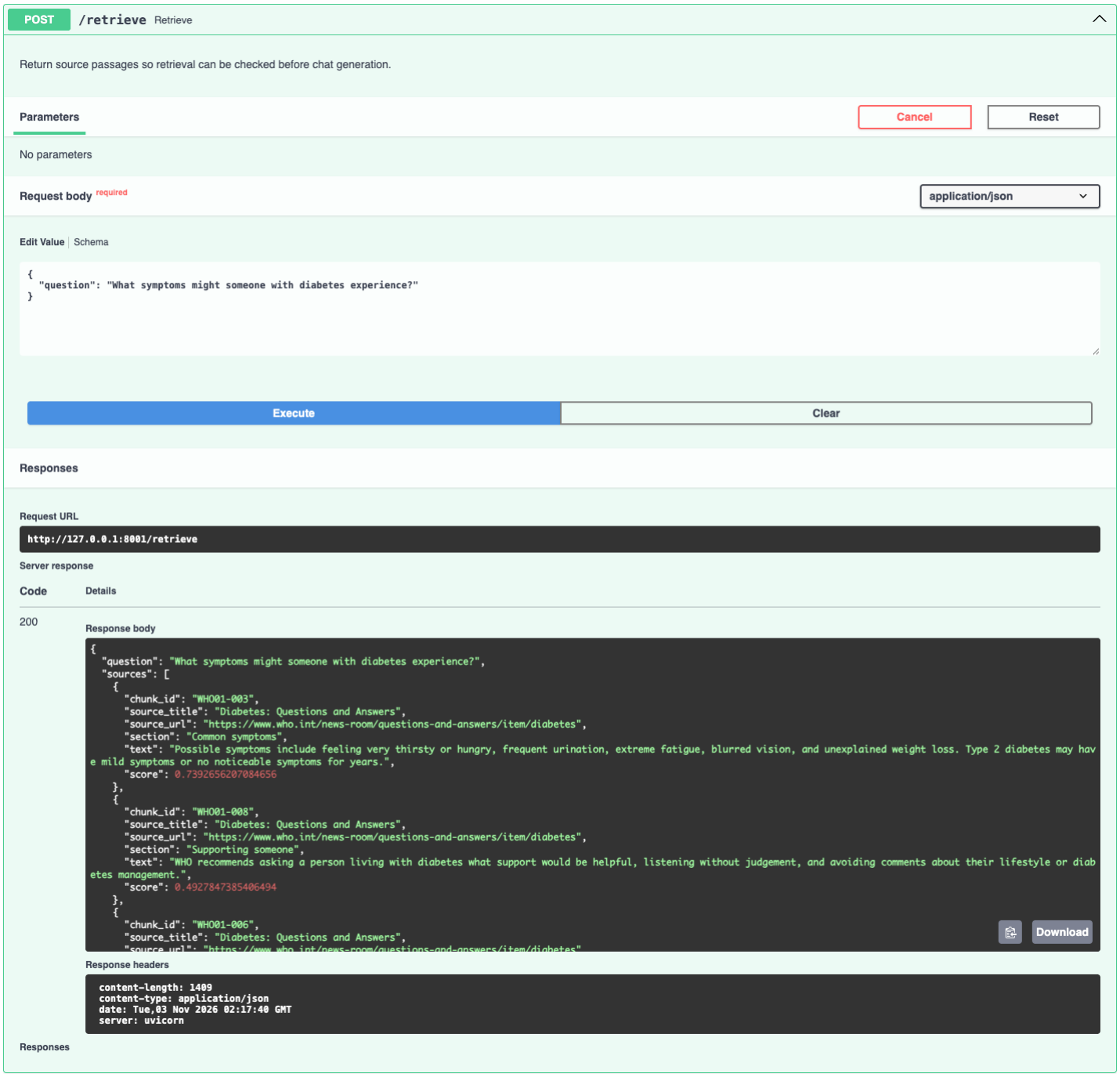


Figure 4: Swagger request and retrieved WHO passages

Figure 5. Swagger response from `POST /chat` for a medication question.

Figure 6. General-information answer and retrieved source sections in the web interface.

Figure 7. Medication question referred to clinic staff in the web interface.

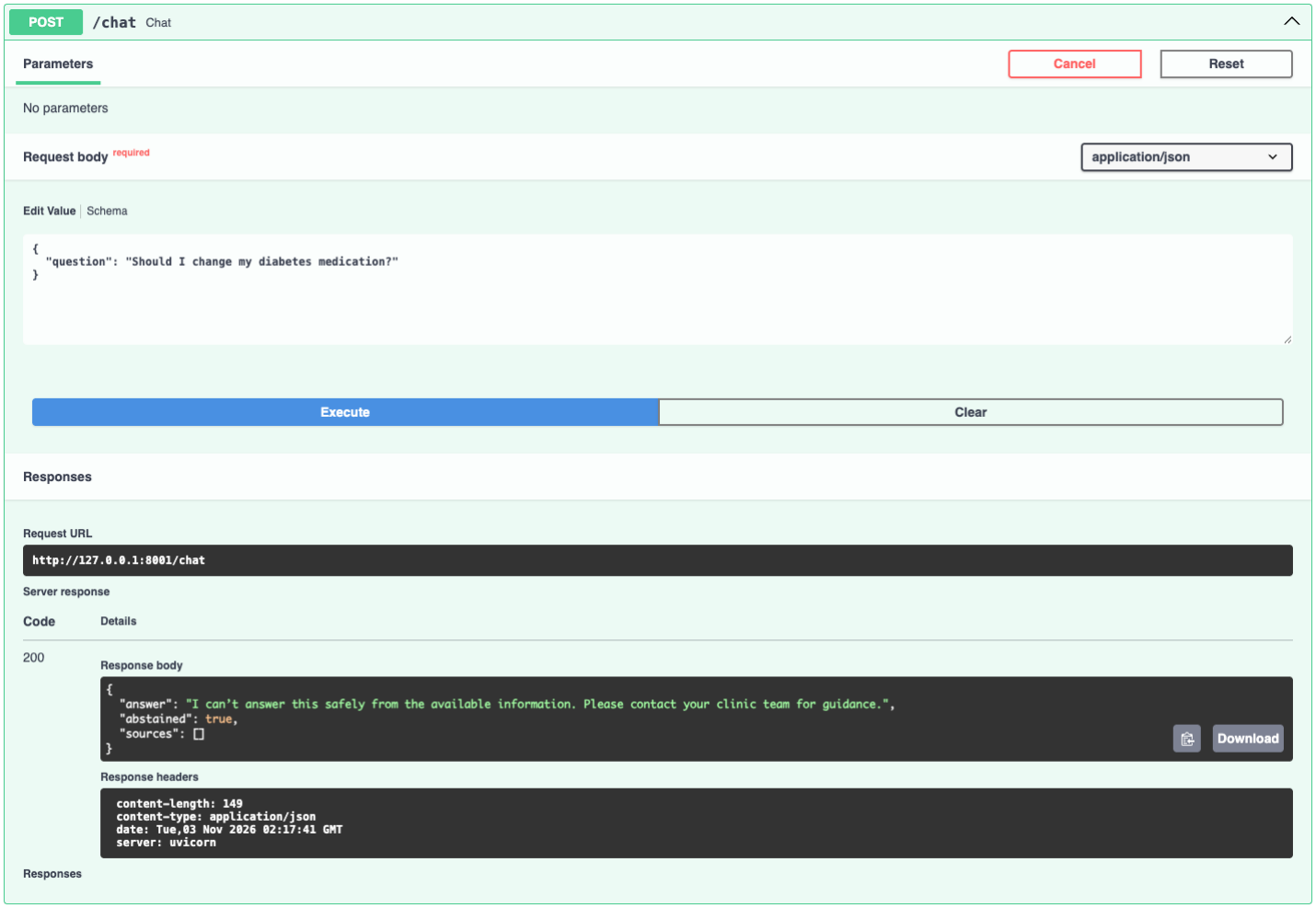


Figure 5: Swagger request and staff referral response

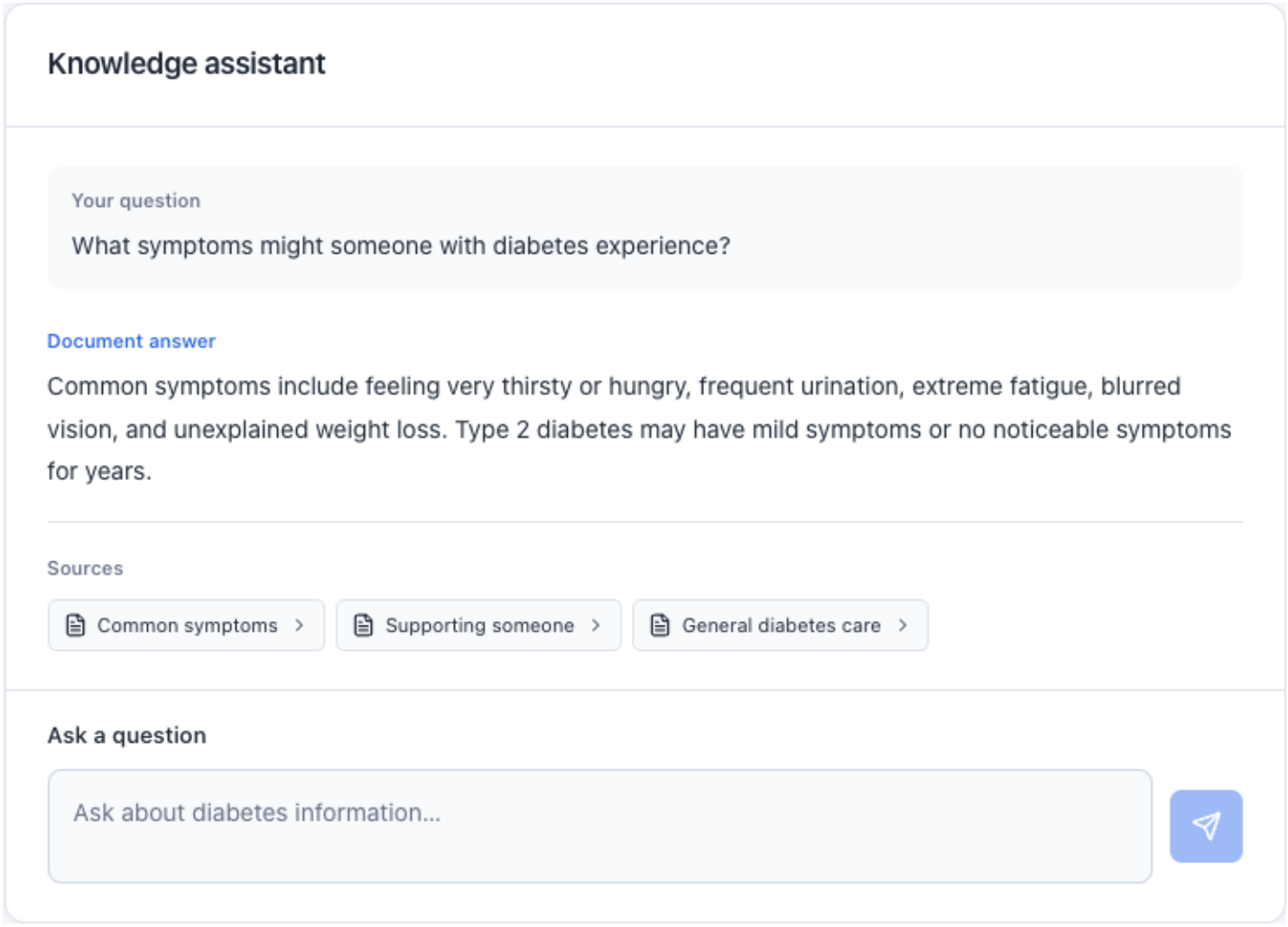


Figure 6: Knowledge assistant answer with retrieved WHO source sections listed

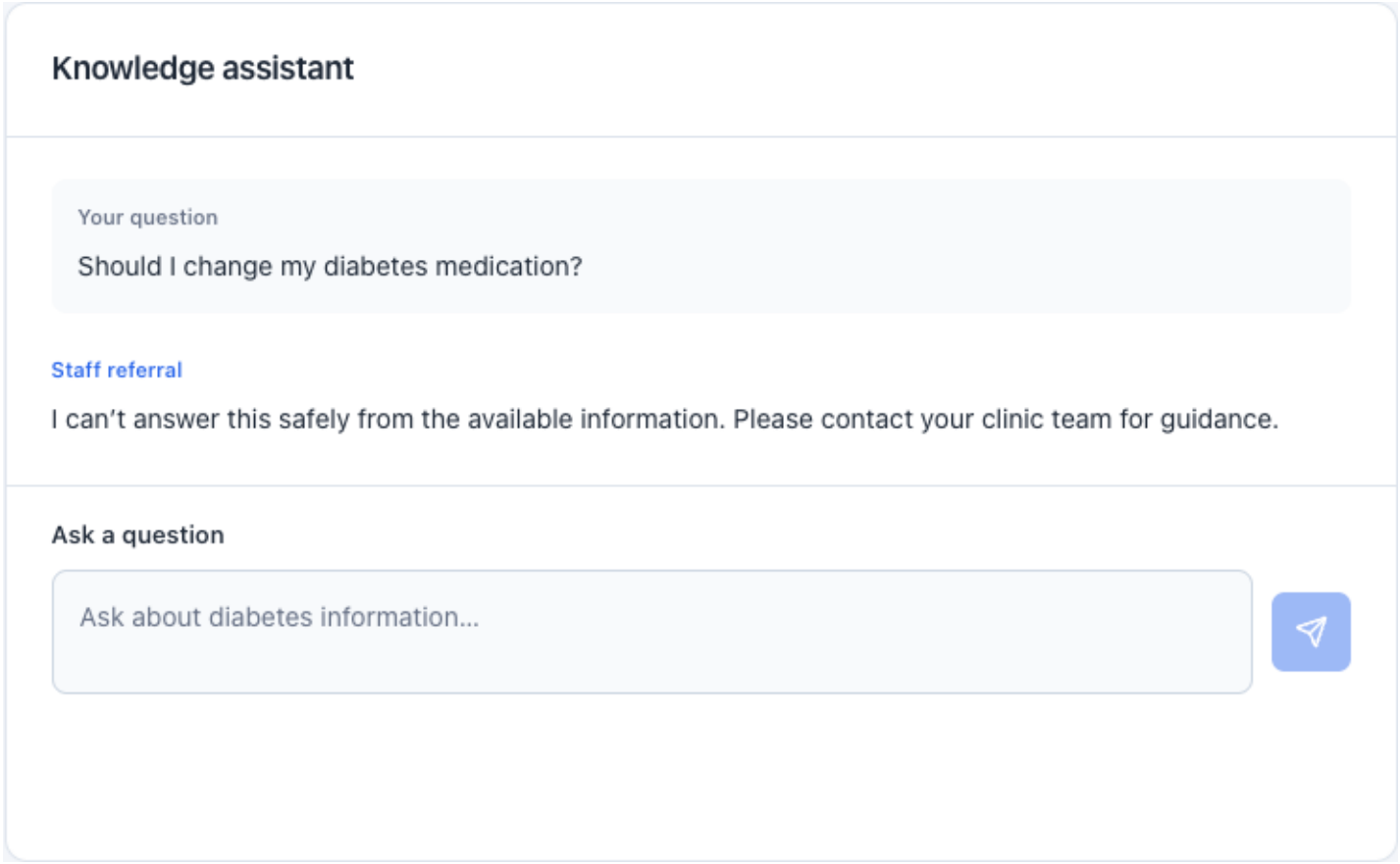


Figure 7: Knowledge assistant staff referral for a medication-change question

Evaluation results

We reviewed 30 RAG cases: 20 questions answerable from the indexed source and 10 expected referrals. We checked retrieval relevance, answer support, citations, completeness, and answer/referral decisions against the WHO passages and answer key. These results are for the initial routing-rule configuration, before we refined the routing rules.

Measure	Result
Retrieval hit@3 on answerable cases	20/20 (100%)
Expected answer/referral decisions matched	23/30 (76.7%)
Expected staff referrals detected	4/10 (40%)
Answerable questions incorrectly abstained	1/20 (5%)
Answerable cases producing an answer judged supported by its source	18/20 (90%)
Answerable cases with a correct source citation	18/20 (90%)
Answerable cases with a correct and complete response	16/20 (80%)

Retrieval returned a relevant passage for all 20 answerable questions. Reviewers found 18 answers supported and 16 both correct and complete. The system made the expected answer or referral decision in 23 of 30 cases. These are baseline results; we did not rerun this evaluation after refining the routing rules.

Current routing regression check

The current application was also checked on 15 existing development cases: eight answerable questions and seven expected referrals. Retrieval found an expected source in all eight answerable cases. The answer/referral decision matched in 14/15 cases (93.3%), and all seven expected referrals were returned. One answerable question was abstained on (1/8). These cases had informed routing development, so this check measures known-case behaviour; the preceding 30-case baseline remains the recorded evaluation of its earlier configuration. Answer support and completeness were not rescored in this regression run.

IV. Situated Learning – Workplace Application

The workplace example is a diabetes-clinic support desk, used as an industry analogue. Its staff review incoming appointment, education, portal, records, and clinical-care messages; prepare handoffs; and consult public education material when answering general questions. Repeated reading and switching between message and reference material can add work and omit details from handoffs. The application brings first-pass classification, summaries, and source passages into one staff workflow. It uses synthetic messages and public WHO material.

Part IV — Bhuvnesh Mishra: Staff Workflow Integration

Anchor — Identify a Workplace Problem Clinic support staff need a single intake workflow for messages that may involve scheduling, education material, portal access, records, or clinical questions. Without a shared intake screen, staff must identify the request, prepare a handoff, and locate information in separate steps. A shared interface is expected to reduce switching between those steps and make the next action easier to see. The diabetes-clinic support desk is a synthetic industry analogue for this workflow.

Apply — Adapt the AI Solution to the Workplace The React staff interface sends support messages and questions to FastAPI. `POST /analyze` returns a category and handoff summary; `POST /retrieve` returns ranked WHO passages; and `POST /chat` returns a source-backed answer or routes selected questions to a staff referral or urgent-care response. Figures 1 and 2 show the request-analysis flow, while Figures 6 and 7 show a source-backed answer and a medication-question referral. The Swagger examples in Figures 1, 4, and 5 document successful API requests.

Analyse — Evaluate Workplace Relevance The API examples returned HTTP 200 for `/metadata`, `/analyze`, `/retrieve`, and `/chat`. In the reviewed outputs, the classifier produced 57 correct categories out of 60; the summary review found every output supported by its input, but all checklist facts were retained in only 10 of 20 cases. The routes connect the tasks in one interface, while the evaluation shows why staff must review the category and summary before using them.

Reflect — Deployment Constraints and Improvements The demonstration uses synthetic messages and public WHO information. A workplace deployment would need privacy and security controls for access, transfer, storage, and retention; review of applicable health-data regulation; approved infrastructure and storage; cost planning; and integration with existing staff systems. The most important improvement for this application perspective is a clinic-approved, access-controlled validation pilot using deidentified representative messages before any connection to scheduling, portal, or records systems. This would check both the API workflow and its outputs with staff while establishing data-handling controls.

Conclusion

We built a React and FastAPI application that brings classification, handoff summaries, and RAG question answering into one diabetes-clinic support workflow. We fully fine-tuned the classifier and evaluated it against the prompted base model on held-out messages. We also reviewed summary fact retention and measured retrieval, answer support, and referral decisions for the recorded RAG baseline. The screenshots show the application and example outputs. Our results show how the three tasks can support staff review in this workflow.